RSA-OAEP: A Proof-of-Concept Implementation with Miller–Rabin Prime Generation

Aleksander Sak, Pratyush Joshi, and Victor Moll Pons

SE-6003 Cryptography, Nanyang Technological University, Singapore

Abstract. Plain ("textbook") RSA is not safe to use on its own because it is deterministic and malleable. RSA-OAEP, standardised in PKCS #1 (RFC 8017), fixes this by adding randomised, hash-based padding before encryption. In this project we implemented RSA-OAEP in Python, writing everything except the hash function ourselves: random prime generation, the Miller–Rabin primality test, key generation, the MGF1 mask function, and OAEP padding. SHA-256 is used by default, but SHA-512, SHA3-256 and SHA3-512 also work. The implementation passes 39 tests and is compatible with the PyCryptodome library. We measured its performance for 1024- to 4096-bit keys and ran some small experiments on prime generation and the Miller–Rabin test. Finally, we point out three weaknesses we found: wasted work in key generation, slow decryption because the CRT is not used, and a timing difference in OAEP decoding that is related to Manger's attack.

Keywords: RSA · OAEP · Miller–Rabin · SHA-2 · SHA-3

1 Introduction

Public-key encryption lets anyone encrypt a message with a public key, while only the owner of the matching private key can decrypt it. RSA [12] was the first practical scheme of this kind and is still widely used, for example in TLS certificates and for exchanging symmetric keys. Its security is based on the fact that multiplying two large primes is easy, but finding them again from the product is very hard.

The basic version of RSA, where the message is simply raised to a power modulo n, has some serious problems. It is deterministic, so the same message always gives the same ciphertext, and an attacker can check guesses such as "yes" or "no". It is also malleable: an attacker can change a ciphertext so that the hidden message changes in a predictable way. In practice the message therefore has to be *padded* before encryption. The Optimal Asymmetric Encryption Padding (OAEP) of Bellare and Rogaway [1], standardised in PKCS #1 [6], adds randomness and structure to the message using a hash function.

In this project we wanted to understand RSA-OAEP by building it ourselves. Apart from the hash function, which comes from Python's standard library, we

implemented all parts: prime generation with the Miller–Rabin test [5,11], key generation, MGF1, and OAEP encoding and decoding. The code is available on GitHub.[1] Section 2 explains the theory, Section 3 describes our implementation, Section 4 presents our results, and Section 5 discusses limitations and open problems.

2 Background

2.1 Textbook RSA

RSA uses arithmetic modulo $n = pq$, where p and q are large primes. Euler's totient is $\varphi(n) = (p-1)(q-1)$, and Euler's theorem says that $a^{\varphi(n)} \equiv 1 \pmod{n}$ for every a coprime to n.

Key generation. Pick two random primes $p \neq q$ of the same size and set $n = pq$. Choose a public exponent e with $\gcd(e, \varphi(n)) = 1$; almost everyone uses $e = 65537$. Then compute $d = e^{-1} \bmod \varphi(n)$ with the extended Euclidean algorithm. The public key is (n, e) and the private key is (n, d).

Encryption and decryption. For a number $0 \leq m < n$:

$$c = m^e \bmod n, \qquad m = c^d \bmod n. \tag{1}$$

This works because $ed = 1 + t\varphi(n)$ for some t, so $c^d = m^{ed} = m \cdot (m^{\varphi(n)})^t \equiv m \pmod{n}$.

Example. Take $p = 61$ and $q = 53$. Then $n = 3233$, $\varphi(n) = 3120$, and with $e = 17$ we get $d = 2753$. The message $m = 65$ encrypts to $c = 65^{17} \bmod 3233 = 2790$, and $2790^{2753} \bmod 3233 = 65$. The example also shows the two problems from the introduction. Encrypting 65 again gives 2790 again (deterministic), and multiplying c by 2^{17} gives a ciphertext that decrypts to $130 = 2m$ (malleable).

2.2 Generating Primes

The prime number theorem says that a random k-bit number is prime with probability about $1/(k \ln 2)$. If we only test odd numbers, we need on average about

$$\frac{k \ln 2}{2} \approx 0.35\, k \tag{2}$$

candidates before finding a prime, i.e. around 355 for a 1024-bit prime [4]. Most candidates have a small factor, so we first do a quick trial division by small primes and only run the slower Miller–Rabin test on the remaining candidates.

[1] `https://github.com/NTU-Cyberboys/SE-6003`

Algorithm 1 Miller–Rabin test with t random bases.

Input: odd n (after trial division), number of rounds t
Output: COMPOSITE or PROBABLY PRIME

write $n-1 = 2^s d$ with d odd
for $i = 1$ **to** t **do**
    pick a random a in $[2, n-2]$
    $x \leftarrow a^d \bmod n$
    if $x = 1$ **or** $x = n-1$ **then continue**
    for $j = 1$ **to** $s-1$ **do**
        $x \leftarrow x^2 \bmod n$
        if $x = n-1$ **then continue** with the next round
    return COMPOSITE
return PROBABLY PRIME

2.3 The Miller–Rabin Test

Fermat's little theorem says that $a^{n-1} \equiv 1 \pmod{n}$ if n is prime. Unfortunately some composite numbers, the Carmichael numbers such as $561 = 3 \cdot 11 \cdot 17$, also pass this test for every a coprime to n. The Miller–Rabin test [5,11] adds a second check based on the fact that modulo a prime the only square roots of 1 are ± 1. We write $n - 1 = 2^s d$ with d odd. If n is prime, then for any base a either $a^d \equiv 1$, or one of $a^d, a^{2d}, \ldots, a^{2^{s-1}d}$ is $\equiv -1 \pmod{n}$. If neither happens, a proves that n is composite (Algorithm 1).

A composite n can still pass a round if we are unlucky with a (such an a is called a *strong liar*). Rabin [11] showed that at most $1/4$ of the bases are strong liars, so after t rounds with random bases the probability of accepting a composite is at most 4^{-t}. With $t = 40$ this is below 2^{-80}. For random candidates the real error is much smaller, which is why standards such as FIPS 186-5 [9] use fewer rounds.

2.4 Hash Functions and MGF1

OAEP needs a hash function H with an output of h_{Len} bytes. We used SHA-2 [7] and SHA-3 [8]. They work very differently inside (SHA-2 iterates a compression function, while SHA-3 is a "sponge" construction), but for OAEP only the interface matters: bytes go in, h_{Len} bytes come out. Because OAEP needs masks longer than h_{Len}, it uses the mask generation function MGF1 [6], which hashes the seed together with a 4-byte counter $0, 1, 2, \ldots$ and concatenates the outputs until it has enough bytes.

2.5 OAEP

Let k be the length of n in bytes, M the message and L an optional label. OAEP encoding (Figure 1) works as follows:

1. Check that $|M| \le k - 2h_{\mathrm{Len}} - 2$.

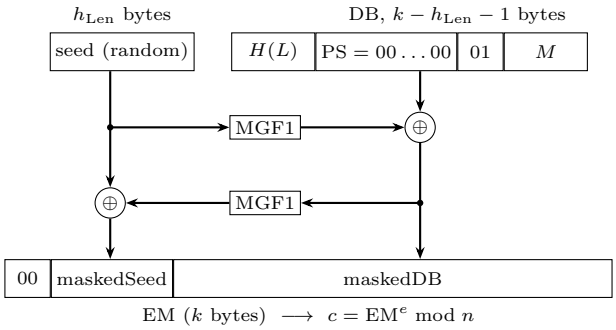


Fig. 1. OAEP encoding as in RFC 8017 [6].

2. Build the data block DB $= H(L) \,\|\, \text{PS} \,\|\, \texttt{01} \,\|\, M$, where PS is enough zero bytes to make DB $k - h_{\text{Len}} - 1$ bytes long.
3. Pick a random seed of h_{Len} bytes.
4. Compute maskedDB $=$ DB $\oplus$ MGF1(seed).
5. Compute maskedSeed $=$ seed $\oplus$ MGF1(maskedDB).
6. The encoded message is EM $=$ `00` $\|$ maskedSeed $\|$ maskedDB, which is then encrypted with textbook RSA.

To decode, the receiver removes the masks in reverse order and checks that the first byte is zero, that $H(L)$ is correct, and that the zeros are followed by `01`. If a ciphertext was tampered with, the result looks random and these checks fail. Because of the random seed, encrypting the same message twice gives different ciphertexts.

RSA-OAEP has been proven secure against chosen-ciphertext attacks in the random oracle model, under the RSA assumption [2,10]. An important practical point is that all decoding errors must look the same to an attacker. Manger [3] showed that if an attacker can tell whether the first byte was zero, for example from an error message or from timing, they can recover the plaintext with roughly $\log_2 n$ queries.

3 Implementation

3.1 Structure

Our implementation is a Python package called `rsaoaep`. It only uses the standard library: `secrets` for random numbers, `hashlib` for SHA-2 and SHA-3, and `hmac` for one constant-time comparison. The rest, including all the number theory and the padding, is our own code. Table 1 shows the modules. Users can call `generate_keypair`, `encrypt` and `decrypt` from Python, or use the command-line tool, which stores keys as JSON files and ciphertexts as base64. On macOS and Linux, private key files are created with owner-only permissions. A demo script encrypts and decrypts some example text, JSON and CSV files, or a text or file given by the user.

Table 1. Modules of the `rsaoaep` package.

Module	What it does
`math_utils.py`	gcd, extended gcd, modular inverse, int/byte conversion
`primality.py`	trial division, Miller–Rabin, prime generation
`rsa_core.py`	key generation, RSA encryption and decryption
`mgf1.py`	MGF1 mask generation function
`oaep.py`	OAEP encoding and decoding
`types.py`, `serialization.py`	key classes; saving and loading keys as JSON
`exceptions.py`	error types
`cli.py`, `benchmark.py`	command-line tool; benchmarks

3.2 Primes and Keys

To generate a k-bit prime, we take k random bits from `secrets.randbits` and set the highest bit (so the number really has k bits) and the lowest bit (so it is odd). The candidate is checked by trial division with the primes up to 37 and then by Algorithm 1 with 40 rounds. If it fails, we try a new random candidate.

`generate_keypair` generates two primes of half the key size each. It starts again if $p = q$, if n is one bit too short, or if $\gcd(e, \varphi(n)) \neq 1$. Then it computes d with our extended Euclidean algorithm. By default we use $e = 65537$ and 2048-bit keys. Keys shorter than 528 bits are rejected, since with SHA-256 the modulus needs at least $2 \cdot 32 + 2 = 66$ bytes. The private key only stores (n, e, d).

3.3 OAEP and Hash Functions

`oaep_encode` follows the steps of RFC 8017, and the random seed comes from the `secrets` module. `decrypt` first checks that the ciphertext has the correct length and is smaller than n, then computes $c^d \bmod n$ and calls `oaep_decode`. The start of the decoding function looks like this:

```
def oaep_decode(encoded: bytes, label: bytes = b"") -> bytes:
    if len(encoded) < 2 * HASH_LENGTH + 2: raise DecryptionError("decryption failed")
    if encoded[0] != 0: raise DecryptionError("decryption failed")
    ... # unmask seed and DB, then check label hash, zero padding and 0x01
```

All errors use the same exception and message, and the label hash is compared in constant time. However, the check of the first byte returns before anything else is computed, which we come back to in Section 4.5.

The hash function is set by one constant, `HASH_NAME = "sha256"`. Since `hashlib` has SHA-2 and SHA-3 with the same interface, we only need to change this one string (e.g. to `"sha3_256"`) to use SHA-3.

3.4 Testing

We have 39 tests (`pytest`), and all of them pass. They cover the number theory functions (e.g. $17^{-1} \bmod 3120 = 2753$ from the example above), primes and composites such as 341 and 561, MGF1, OAEP with the longest allowed message and one byte more, round trips through JSON key files, the CLI and the demo, file permissions of private keys, and invalid inputs such as wrong labels, changed

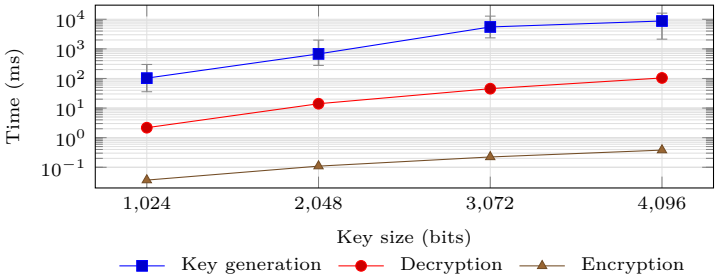


Fig. 2. Median times (log scale). The bars show the fastest and slowest key generation.

ciphertexts and too-small key sizes. As RFC 8017 has no SHA-256 test vectors for OAEP, we also checked that our code works together with the PyCryptodome library in both directions.

4 Results

All measurements were done on one machine (a Linux aarch64 virtual machine running on one of our Apple-silicon laptops, Python 3.10). The script `report/experiments/report_experiments.py` in our repository repeats all of them.

4.1 Correctness

For SHA-256, SHA-512, SHA3-256 and SHA3-512 we encrypted 50 random messages with random labels using a 2048-bit key, both with our code and with PyCryptodome, and decrypted each with the other library. All 400 round trips worked. The longest message that fits is 190 bytes for the 256-bit hashes and 126 bytes for the 512-bit hashes. OAEP encoding took about 11–12 µs for every hash, so the choice between SHA-2 and SHA-3 does not really matter for speed. When we encrypted the same message 1000 times, we got 1000 different ciphertexts.

4.2 Performance

Figure 2 and Table 2 show our timings. Encryption is very fast (below 0.4 ms) because $e = 65537$ is small. Decryption uses the large exponent d and takes from 2.2 ms (1024 bits) up to 104 ms (4096 bits). Key generation is by far the slowest operation, and its time varies a lot, because the number of candidates needed to find a prime is random. At 4096 bits a single key took anywhere between 2.1 and 16.1 seconds.

Table 2. Median times in milliseconds. Key generation was repeated N times; encryption and decryption 200 times (60 times for 4096 bits).

Key size	N	Key generation	Encryption	Decryption
1024	30	102.7	0.037	2.17
2048	20	675.7	0.109	14.02
3072	10	5497.2	0.223	45.19
4096	12	8744.7	0.380	104.14

Table 3. Candidates needed per prime, compared with $k \ln 2/2$.

Bits k	Primes	Average candidates	Predicted	Removed by trial division
512	300	189.3	177.4	70.0%
1024	150	353.7	354.9	70.2%
1536	60	642.6	532.3	70.3%

4.3 Prime Generation

We counted how many candidates were needed to find each prime (Table 3). The averages are close to the prediction from Equation (2), for example 354 versus 355 for 1024-bit primes. (The 1536-bit average is a bit higher, but we only had 60 samples and the spread is very large.) About 70% of the candidates were removed by trial division. None of the composites that reached Miller–Rabin survived even the first round, so most of the testing time is spent on the 40 rounds for the prime that is finally accepted (about 85 ms for a 1024-bit prime).

While doing this, we noticed that key generation often throws away a pair of primes. Since only the highest bit of p and q is set, their product sometimes has one bit too few, and the pair has to be generated again. If p and q are roughly uniform, this happens with probability $2 \ln 2 - 1 \approx 39\%$. We measured 37–38% (200 keys at 1024 bits and 40 keys at 2048 bits), so on average key generation does about 60% more work than needed.

4.4 Miller–Rabin Error Rates

We also wanted to see how close real numbers come to the bound of 1/4 strong liars. For some well-known composites we counted the strong liars among all bases $a \in [2, n-2]$, or sampled 200,000 random bases for the large ones (Table 4). The Carmichael number 561 fools the Fermat test for 318 bases but Miller–Rabin for only 8. The two large numbers in the table pass Miller–Rabin for all prime bases up to 7 and 23 respectively, so a version with these fixed bases would accept them as prime. For them the fraction of liars is basically exactly 1/4. Our implementation, which uses random bases, still rejected both numbers in 1000 out of 1000 tries.

4.5 Timing of Decoding Errors

Because of the early return mentioned in Section 3, we checked whether different decoding errors take different amounts of time. We made three 2048-bit cipher-

Table 4. Fraction of bases that are strong liars. †Estimated from 200,000 random bases.

n	Note	Strong liars
561	Carmichael number ($3 \cdot 11 \cdot 17$)	1.4%
2047	fools base 2 ($23 \cdot 89$)	11.7%
1373653	fools bases 2, 3 ($829 \cdot 1657$)	18.7%
3215031751†	fools bases 2, 3, 5, 7	24.8%
3825123056546413051†	fools all prime bases up to 23	25.0%

texts: one where the decrypted first byte is not zero, one where it is zero but the rest is random (so the label check fails), and a valid one. `oaep_decode` alone took 0.17 µs for the first, 11.0 µs for the second and 14.5 µs for the valid one, so the first type of error is clearly faster. When timing the whole `decrypt` function, the difference is only about 20 µs out of 14.6 ms, which is smaller than the normal variation between runs (about 170 µs). With enough repeated measurements an attacker could probably still see the difference, which is exactly the information Manger's attack [3] needs.

5 Discussion

The implementation works correctly and follows the standard, which we confirmed by testing it against PyCryptodome with both SHA-2 and SHA-3. Our measurements also match the theory quite well. Still, it is a proof of concept and should not be used for real data. The main problems we found are:

- *Timing leak in decoding.* The first-byte check should not return early. All checks should be computed and combined at the end.
- *Python arithmetic is not constant time.* Python's `pow` can leak information about d through timing. Real libraries use constant-time arithmetic and *blinding* (decrypting $c \cdot r^e$ for a random r and removing r afterwards).
- *No CRT.* Because p and q are not stored, we cannot use the Chinese Remainder Theorem for decryption. In a quick test this made decryption about 3.5× faster (4.0 instead of 14.0 ms at 2048 bits).
- *Wasted primes.* Setting the two highest bits of p and q would avoid throwing away 39% of the prime pairs. FIPS 186-5 [9] also requires p and q not to be too close to each other, which we do not check, and allows fewer Miller–Rabin rounds than our 40.
- *Key storage.* Private keys are saved as unencrypted JSON files.

RSA-OAEP can only encrypt short messages (190 bytes for a 2048-bit key with SHA-256). In practice it is used to encrypt a random symmetric key, which then encrypts the actual data. Our demo only encrypts small files for this reason.

Open problems. The security proof of OAEP only works in the random oracle model, where the hash function is treated as a perfectly random function, and

the proof is not tight [10]. A more practical problem is that Shor's algorithm [13] could factor RSA moduli on a large quantum computer. Because of this, RSA is expected to be replaced by post-quantum schemes in the long run.

6 Conclusion

We implemented RSA-OAEP from scratch, including prime generation with Miller–Rabin, key generation, MGF1 and OAEP with SHA-2 or SHA-3. The implementation passes all tests and works together with PyCryptodome. Our experiments agreed with the theory: the number of candidates per prime matched the prime number theorem, and the worst composites reached the Miller–Rabin bound of 1/4. We also found some weaknesses, mainly a timing difference in OAEP decoding. The project showed us that a correct implementation is not automatically a secure one, and that details such as padding and error handling are as important as the mathematics behind RSA.

References

1. Bellare, M., Rogaway, P.: Optimal asymmetric encryption. In: De Santis, A. (ed.) Advances in Cryptology—EUROCRYPT '94. Lecture Notes in Computer Science, vol. 950, pp. 92–111. Springer (1994). https://doi.org/10.1007/BFb0053428
2. Fujisaki, E., Okamoto, T., Pointcheval, D., Stern, J.: RSA-OAEP is secure under the RSA assumption. In: Advances in Cryptology—CRYPTO 2001. Lecture Notes in Computer Science, vol. 2139, pp. 260–274. Springer (2001). https://doi.org/10.1007/3-540-44647-8_16
3. Manger, J.: A chosen ciphertext attack on RSA optimal asymmetric encryption padding (OAEP) as standardized in PKCS #1 v2.0. In: Advances in Cryptology—CRYPTO 2001. Lecture Notes in Computer Science, vol. 2139, pp. 230–238. Springer (2001). https://doi.org/10.1007/3-540-44647-8_14
4. Menezes, A.J., van Oorschot, P.C., Vanstone, S.A.: Handbook of Applied Cryptography. CRC Press (1996)
5. Miller, G.L.: Riemann's hypothesis and tests for primality. Journal of Computer and System Sciences **13**(3), 300–317 (1976). https://doi.org/10.1016/S0022-0000(76)80043-8
6. Moriarty, K., Kaliski, B., Jonsson, J., Rusch, A.: PKCS #1: RSA cryptography specifications version 2.2. RFC 8017, Internet Standard (2016). https://doi.org/10.17487/RFC8017
7. National Institute of Standards and Technology: Secure hash standard (SHS). FIPS PUB 180-4, NIST (2015). https://doi.org/10.6028/NIST.FIPS.180-4
8. National Institute of Standards and Technology: SHA-3 standard: Permutation-based hash and extendable-output functions. FIPS PUB 202, NIST (2015). https://doi.org/10.6028/NIST.FIPS.202
9. National Institute of Standards and Technology: Digital signature standard (DSS). FIPS PUB 186-5, NIST (2023). https://doi.org/10.6028/NIST.FIPS.186-5
10. Pointcheval, D.: How to encrypt properly with RSA. CryptoBytes **5**(1), 10–19 (2002), rSA Laboratories
11. Rabin, M.O.: Probabilistic algorithm for testing primality. Journal of Number Theory **12**(1), 128–138 (1980). https://doi.org/10.1016/0022-314X(80)90084-0

12. Rivest, R.L., Shamir, A., Adleman, L.: A method for obtaining digital signatures and public-key cryptosystems. Communications of the ACM **21**(2), 120–126 (1978). https://doi.org/10.1145/359340.359342
13. Shor, P.W.: Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. SIAM Journal on Computing **26**(5), 1484–1509 (1997). https://doi.org/10.1137/S0097539795293172

A Reproducing the Results

From the repository root (Python 3.9 or newer; use `python` instead of `python3` in an activated Windows environment):

```
python3 -m pip install -e ".[dev]"
python3 -m pytest # 39 tests
python3 scripts/run_demo.py # demo with example files
python3 -m pip install pycryptodome && python3 scripts/reference_validation.py
PYTHONPATH=src python3 report/experiments/report_experiments.py \
    env random hashes keygen:1024 keygen:2048 mr timing crt \
    prime:512 prime:1024 prime:1536 bench:1024 bench:2048 bench:3072 bench:4096
```

The results are written to `report/experiments/results_*.json`.

B Additional Measurements

Second machine. We also ran our benchmark script (`run_benchmarks.py`, 3 repeats) directly on macOS (arm64, Python 3.15). The median times for 1024, 2048 and 3072 bits were 48.9, 1100.7 and 3635.1 ms for key generation, 0.071, 0.153 and 0.313 ms for encryption, and 2.38, 16.21 and 53.81 ms for decryption. Decryption is within 10–20% of Table 2. Encryption is slower there because that script times only one encryption per key instead of taking the median of 200 runs.

Mean and standard deviation of key generation (ms): 126.9 ± 72.4 (1024 bits), 879.0 ± 520.4 (2048), 6409.5 ± 3445.7 (3072) and 8600.2 ± 4564.6 (4096).

More Miller–Rabin data. Other composites we checked (strong-liar fraction): 341: 14.2%, 703: 22.9%, 1729: 9.3%, 8911: 20.0%. A single Miller–Rabin round rejected all 20,000 random 512-bit composites without small factors that we tried.